
B28: SQL SuperHero Program : Session 36

1 message

Vishal Jaiswal <vishaldbdev@gmail.com>
Bcc: satishhattekar1997@gmail.com

25 April 2025 at 22:57

Can we commit Inside Trigger ?Most common question in Interview.

YES, we can do the commit inside the trigger but for this we have to make that transaction independent of the parent transaction. It means, we have to use pragma autonomous transactions to make this happen. In the Normal scenario, we can not.

If we create a trigger using commit inside, it does not give any error while trigger creation. It will Run time error when that trigger is called.

```
create table tab1 (c1 number)
```

→ Table created

Table created.

```
create table tab1_log (timedata date, operation varchar2(20))
```

Table created.

→ Table for logging
Auditing

```
create or replace trigger mytrigger
```

```
after insert on tab1
```

```
begin
```

```
insert into tab1_log values (sysdate,'Insert done');  
Commit;  
End;
```

See the above code of trigger , There is no compilation issue. Trigger will create successfully.

```
create or replace trigger mytrigger  
after insert on tab1  
begin  
    insert into tab1_log values (sysdate,'Insert done');  
Commit;  
End;
```

→ We wrote commit

Trigger created.

→ Trigger created

---> **Mention this point in Interview that we can even write COMMIT in trigger without autonomous_transaction and there is no issue, trigger will be created, compiled, but yes it gives issue when that trigger is fired.**

```
create or replace trigger mytrigger  
after insert on tab1  
begin  
    insert into tab1_log values (sysdate, 'insert completed');  
    commit;  
End;
```

Trigger created.

```
insert into tab1 values (100)
```

→ Here it fails (Run time error)

ORA-04092: cannot COMMIT in a trigger
ORA-06512: at "SQL_VXINXFHHWBIWVMOOHTTXVIROR.MYTRIGGER", line 3
ORA-06512: at "SYS.DBMS_SQL", line 1721

Lets make this autonomous.

create or replace trigger mytrigger

after insert on tab1

declare

pragma autonomous_transaction;

begin

insert into tab1_log values (sysdate, 'Insert done');

commit;

end;

```
create or replace trigger mytrigger
```

```
after insert on tab1
```

```
    declare
```

```
        pragma autonomous_transaction;
```

→ Added

```
Begin
```

```
    insert into tab1_log values (sysdate, 'Insert done');
```

```
    commit;
```

```
End;
```

```
Trigger created.
```

```
insert into tab1 values (100);
```

```
select * from tab1
```

C1
100

→ inserted here

```
select * from tab1_log
```

TIMEDATA	OPERATION
24-APR-25	Insert done

→ And also here automatically

--> Remember that the trigger logging will be committed even if the main transaction will be rolled back.

Suppose we did something and we rollbacked, but if that is autonomous written in trigger, that will not be rollbacked.

```
declare
```

```
l_count number;
```

```
Begin
```

```
insert into TAB1 values (200);
```

```
Select count(*) Into l_count from tab1_log;
```

```
dbms_output.put_line(l_count);
```

```
Rollback;
```

```
End;
```

```
Statement processed.
```

```
2
```

→ insert in table

→ Because of this trigger fired and insert in log table & commit

→ NO impact on log table, because program - Autonomous

```
select * from tab1
```

C1
100

→ we inserted 200 &
its rollback

```
select * from tab1_log
```

TIMEDATA	OPERATION
24-APR-25	Insert done
24-APR-25	Insert done

→ but data committed
here bcoz of
Pragma

2 rows selected.

Mutating Trigger

Mutates means, it is changing. Something that is changing is hard to analyze and to quantify. A mutating table error (ORA-04091) occurs when a row-level trigger tries to examine or change a table that is already undergoing change (via an INSERT, UPDATE, or DELETE statement). In particular, this error occurs when a row-level trigger attempts to read or write the table from which the trigger was fired. Fortunately, the same restriction does not apply in statement-level triggers.

We discussed triggers, but just to recall, Here are some important items to remember about triggers.

- On insert triggers have no :OLD values.
- On delete triggers have no :NEW values.
- Triggers do not commit transactions. If a transaction is rolled back, the data changed by the trigger is also rolled back.
- Commits, rollbacks and save points are not allowed in the trigger body. A commit/rollback affects the entire transaction, it is all or none.
- Unhandled exceptions in the trigger will cause a rollback of the entire transaction, not just the trigger.
- If more than one trigger is defined on an event, the order in which they fire is not defined. If the triggers must fire in order, you must create one trigger that executes all the actions in the required order.
- A trigger can not change a table that it has read from. This is the mutating table error issue.

A mutation table is defined as a table that is changing. Mutating error normally occurs when we are performing DML operations and the trigger fired and we are trying to select the affected record from the same trigger. So, basically we are trying to select the record in the trigger from the table who owns the trigger. This creates consistency and throw error, which is called mutating error.

When we talk about a solution, please do not talk about compound triggers directly. Every knows that it is written on the internet. Try to discuss solutions.

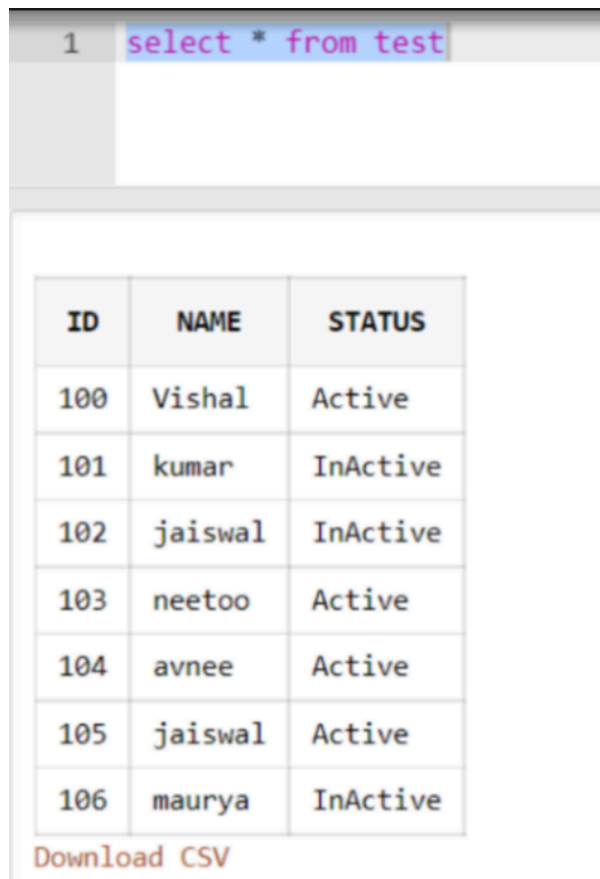
1. Do Not use triggers - The best way to avoid the mutating table error is not to use triggers. Most PL/SQL developers avoid triggers unless absolutely necessary. As it impacts performance as well
2. If possible can we use Statement level, because mutating comes only with Row Level.
3. From Oracle 11g Onwards, we have a Compound Trigger to avoid mutating tables with a combination of row-level and statement-level triggers.

4. Use autonomous transactions - You can avoid the mutating table error by marking your trigger as an autonomous transaction, making it independent from the table that calls the procedure.

=====

```
create table test (id number, name varchar2(20), status varchar2(20));
```

```
Insert into test values (100, 'Vishal', 'Active');
Insert into test values (101, 'kumar', 'InActive');
Insert into test values (102, 'jaiswal', 'InActive');
Insert into test values (103, 'neetoo', 'Active');
Insert into test values (104, 'avnee', 'Active');
Insert into test values (105, 'jaiswal', 'Active');
Insert into test values (106, 'maurya', 'InActive');
```



1 select * from test

ID	NAME	STATUS
100	Vishal	Active
101	kumar	InActive
102	jaiswal	InActive
103	neetoo	Active
104	avnee	Active
105	jaiswal	Active
106	maurya	InActive

Download CSV

Let's Create Trigger :

Create or replace trigger MutatingTrigger_Test

After Update on test for each row

declare

 v_count number;

BEGIN

 select count(*) into v_count from test where status = 'Active';

 dbms_output.put_line('Total number of active '||v_count);

End;

```
1 Create or replace trigger MutatingTrigger_Test
2 After Update on test for each row
3 declare
4 v_count number;
5 BEGIN
6     select count(*) into v_count from test where status = 'Active';
7     dbms_output.put_line('Total number of active '||v_count);
8 End;
```

Row level

Trigger created.

Update test set Status = 'Active' where id in (101,102)

mutating

ORA-04091: table SQL_ZCZOILCMCDOWUEOHJFNJSHPLQ.TEST is mutating, trigger/function may not see it
ORA-06512: at "SYS.DBMS_SQL", line 1721

Solution 1.:

Do Not create trigger and make some logic.

Solution 2.:

Create a statement level trigger. Mutating only comes with Row level trigger.

```
create or replace trigger mutatingtrigger_test
```

```
after update on test -- for each row
```

```
  Declare
```

```
  v_count number;
```

```
  Begin
```

```
    select count(*) into v_count from test where status = 'Active';
```

```
    dbms_output.put_line(v_count);
```

```
  End;
```

```
Trigger created.
```

→ just comment this and it becomes statement level

```
update test set status = 'Active' where id in (101,102)
```

```
2 row(s) updated.
```

```
6
```

✓ ① No error
② Trigger fired

Solution 3:

Make the Transaction Independent using Pragma Autonomous transaction feature. As we know Pragma is very powerful and it is totally independent execution.

```
1 Create or replace trigger MutatingTrigger_Test
2 After Update on test for each row → its row level
3 declare
4     v_count number;
5     Pragma Autonomous_Transaction; → its pragma
6 BEGIN
7     select count(*) into v_count from test where status = 'Active';
8     dbms_output.put_line('Total number of active ' || v_count);
9 End;
```

```
Trigger created.
```



```
update test set status = 'Active' where id in (101,102)
```

```
2 row(s) updated.
```

```
6
```

```
6
```

=====

=====

Solution: 4

Lets create Compound trigger. It introduced in Oracle 11g and it makes things easy.

Create or replace trigger MutatingTrigger_Test

For Update on test

Compound Trigger

```
        v_count number;
After Each Row Is
BEGIN
    dbms_output.put_line('Updation Done');
End After Each Row;

After Statement Is
Begin
    select count(*) into v_count from test where status = 'Active';
    dbms_output.put_line('Total number of active '||v_count);
End After Statement ;
End MutatingTrigger_Test;
```

Before/After Replace by FOR

```
1 Create or replace trigger MutatingTrigger_Test
2 For Update on test
3 Compound Trigger
4
5     v_count number;
6 After Each Row Is
7     BEGIN
8         dbms_output.put_line('Updation Done');
9     End After Each Row;
10
11     After Statement Is
12     Begin
13         select count(*) into v_count from test where status = 'Active';
14         dbms_output.put_line('Total number of active '||v_count);
15     End After Statement ;
16 End MutatingTrigger_Test;
```

Declare Compound

Row level

Statement level

Trigger created.

```
1 Update Test Set Status = 'Inactive' Where Id In (102,101)
```

Done.

2 row(s) updated.

Updation Done

Updation Done

Total number of active 4